Materia: Programación IV

UNIDAD 6 · RELACIONES Y ESTADO

APUNTE TEÓRICO

JSON Server y consumo de APIs REST

Revisor de la unidad: Prof. Juan Ignacio Sarmiento

En el desarrollo de aplicaciones frontend modernas es común que el equipo de backend aún no haya finalizado la API o que el frontend necesite comenzar su desarrollo de manera independiente. Para resolver este problema se utilizan herramientas que permiten **simular un servidor REST** de forma rápida.

Una de las herramientas más utilizadas para este propósito es **JSON Server**.

JSON Server es una herramienta basada en **Node.js** que permite generar una API REST completa a partir de un simple archivo JSON. De esta manera, el desarrollador puede trabajar con endpoints reales sin necesidad de implementar un backend completo.

Esto hace posible:

- Simular un servidor real durante el desarrollo.
- Probar operaciones CRUD (crear, leer, actualizar y eliminar).
- Desarrollar interfaces que consumen datos desde una API.
- Trabajar con datos persistentes durante las pruebas.

Por estas razones, JSON Server es muy utilizado en contextos educativos y en las primeras etapas de desarrollo de una aplicación.

Instalación y puesta en marcha

Para utilizar JSON Server es necesario tener instalado **Node.js**, ya que la herramienta se ejecuta dentro del ecosistema de JavaScript.

La instalación puede realizarse mediante el gestor de paquetes **npm**, o con **pnpm**, que es el gestor empleado en los videos de la actividad. Los dos comandos son equivalentes.

Ejemplo de instalación:

```
npm install json-server@0.17.4
# equivalente con pnpm:
pnpm add -D json-server@0.17.4
```

VERSIÓN DE JSON SERVER

Se fija la versión `0.17.4`, la estable. Sin versión, `npm` instala la `1.0.0-beta`, que trata todos los `id` como texto: devuelve `"id": "1"` en lugar de `"id": 1`, reescribe así el `db.json` y genera los registros nuevos con identificadores como `"89-XieqKUQk"`. Con esa versión, una comparación como `producto.id === 1` deja de funcionar. Los ejemplos de este apunte usan `id` numéricos.

Una vez instalada la herramienta, el siguiente paso consiste en crear un archivo que actúa como base de datos simulada. Este archivo normalmente se denomina `db.json`.

Posteriormente se inicia el servidor utilizando un comando que indica qué archivo utilizar como base de datos.

```
npx json-server db.json
# equivalente con pnpm:
pnpm json-server db.json
```

Una vez iniciado, el servidor queda disponible normalmente en:

```
http://localhost:3000
```

Desde ese momento, el archivo JSON se comporta como una base de datos y puede ser consultado mediante peticiones HTTP.

Estructura de la base de datos simulada

El funcionamiento de JSON Server se basa en la estructura de un archivo JSON que representa los recursos de la aplicación.

Cada propiedad del objeto principal representa una **colección de datos**, similar a una tabla dentro de una base de datos.

Ejemplo conceptual de estructura:

```
{
  "categorias": [
    {
      "id": 1,
      "nombre": "Electrónica"
    }
  ],
  "productos": [
    {
      "id": 1,
      "nombre": "Laptop",
      "categoriaId": 1
    }
  ]
}
```

En este ejemplo se definen dos colecciones:

- **categorias**
- **productos**

Cada una contiene una lista de objetos con sus respectivos atributos.

A partir de esta estructura, JSON Server genera automáticamente una serie de **endpoints REST**.

Endpoints generados automáticamente

Una de las principales ventajas de JSON Server es que crea automáticamente rutas para interactuar con los datos.

Estas rutas siguen la estructura estándar de una API REST.

Método HTTP	Endpoint	Función
GET	`/categorias`	Obtener todas las categorías
GET	`/categorias/1`	Obtener una categoría específica
POST	`/categorias`	Crear una nueva categoría
PUT / PATCH	`/categorias/1`	Actualizar una categoría
DELETE	`/categorias/1`	Eliminar una categoría

Así se simula completamente el comportamiento de una API real sin escribir lógica de backend.

Relaciones entre datos en JSON Server

En muchas aplicaciones los datos no son independientes. Es común que exista una relación entre distintas entidades.

Por ejemplo:

- Un producto pertenece a una categoría.
- Un comentario pertenece a una publicación.
- Un pedido pertenece a un usuario.

En JSON Server estas relaciones se representan mediante **identificadores de referencia**.

En el ejemplo anterior, cada producto posee un campo llamado `categoriaId`. Este campo indica a qué categoría pertenece el producto.

Conceptualmente el modelo funciona de la siguiente manera:

```
{
  "categorias": [
    { "id": 1, "nombre": "Electrónica" },
    { "id": 2, "nombre": "Hogar" }
  ],
  "productos": [
    { "id": 1, "nombre": "Laptop", "categoriaId": 1 },
    { "id": 2, "nombre": "Heladera", "categoriaId": 2 }
  ]
}
```

Aquí se puede observar que cada producto contiene un identificador que referencia a una categoría existente.

Con este mecanismo se construyen relaciones simples entre entidades, similares a las claves foráneas en bases de datos relacionales.

Consumo de API con Datos Asociados

Una vez que el servidor de **JSON Server** está en funcionamiento, la aplicación frontend puede comenzar a interactuar con los datos mediante **peticiones HTTP**.

Este proceso se conoce como **consumo de API** y consiste en enviar solicitudes al servidor para:

- Obtener información
- Crear nuevos registros
- Modificar datos existentes
- Eliminar registros

Estas operaciones siguen el estándar de las **APIs REST**, utilizando distintos métodos HTTP.

Métodos HTTP utilizados en APIs REST

En una API REST cada operación se realiza mediante un método HTTP específico. Cada método representa una acción sobre los datos del servidor.

Método	Función
GET	Obtener datos del servidor
POST	Crear un nuevo registro
PUT / PATCH	Actualizar datos existentes
DELETE	Eliminar un registro

Estos métodos permiten implementar lo que se conoce como **operaciones CRUD**.

CRUD es un acrónimo que representa las operaciones básicas que pueden realizarse sobre los datos:

- **Create** → crear datos
- **Read** → leer datos
- **Update** → actualizar datos
- **Delete** → eliminar datos

Obtener datos de la API (GET)

La operación más común es la obtención de datos desde el servidor. Esto se realiza mediante el método **GET**.

Cuando una aplicación React realiza una petición GET, el servidor responde enviando los datos en formato JSON.

Ejemplo conceptual de solicitud:

```
fetch("http://localhost:3000/productos")
  .then((response) => response.json())
  .then((productos) => setProductos(productos))
```

Esta petición solicita al servidor la lista de productos almacenados en la base de datos simulada.

Una vez recibidos los datos, la aplicación normalmente los guarda en el **estado del componente** para poder renderizarlos en la interfaz (en el ejemplo, con la función `setProductos` de un `useState`).

Este proceso posibilita mostrar información dinámica proveniente del servidor.

Crear nuevos registros (POST)

El método **POST** se utiliza cuando se desea agregar nuevos datos al servidor.

En una aplicación real esto ocurre, por ejemplo, cuando el usuario completa un formulario y presiona un botón de guardar.

En ese momento, el frontend envía los datos al servidor para que sean almacenados.

Ejemplo de creación:

```
fetch("http://localhost:3000/productos", {
  method: "POST",
  body: JSON.stringify(nuevoProducto),
  headers: {
    "Content-Type": "application/json"
  }
})
```

El servidor recibe la información y crea un nuevo registro dentro de la colección correspondiente.

Actualizar datos existentes (PUT / PATCH)

Cuando se necesita modificar información que ya existe en el servidor se utilizan los métodos **PUT** o **PATCH**.

Estos métodos permiten actualizar registros específicos utilizando su identificador.

Por ejemplo, si se desea modificar un producto con id 1, la solicitud se realiza hacia el endpoint correspondiente.

Ejemplo de actualización:

```
fetch("http://localhost:3000/productos/1", {
  method: "PUT",
  body: JSON.stringify(productoActualizado),
  headers: {
    "Content-Type": "application/json"
  }
})
```

La diferencia conceptual entre ambos métodos es:

- **PUT** reemplaza completamente el objeto.

- **PATCH** modifica solo algunos campos.

En muchos casos JSON Server permite utilizar cualquiera de los dos métodos para actualizar registros.

Eliminar registros (DELETE)

El método **DELETE** se utiliza para eliminar un registro existente del servidor.

Este tipo de operación suele utilizarse en interfaces administrativas donde el usuario puede borrar elementos de una lista.

Ejemplo de eliminación:

```
fetch("http://localhost:3000/productos/1", {
  method: "DELETE"
})
```

Una vez realizada la operación, el servidor elimina el registro correspondiente de la base de datos simulada.

Posteriormente el frontend suele actualizar su estado para reflejar el cambio en la interfaz.

Flujo general de consumo de APIs en una aplicación React

Cuando una aplicación React consume una API, normalmente sigue el siguiente flujo:

1. El componente realiza una solicitud HTTP al servidor.
2. El servidor procesa la solicitud y accede a los datos.
3. La API devuelve una respuesta en formato JSON.
4. React guarda los datos recibidos en el estado.
5. La interfaz se actualiza mostrando la información.

Este modelo constituye la base del desarrollo de aplicaciones frontend conectadas a servicios externos.

Bibliografía

1. Banks, A., & Porcello, E. (2020). Learning React (2.ª ed.). O'Reilly Media.
2. Cherny, B. (2019). Programming TypeScript. O'Reilly Media.
3. Microsoft. (s. f.). TypeScript handbook. Recuperado el 2 de octubre de 2026, de https://www.typescriptlang.org/docs/handbook
4. React. (s. f.). React documentation. Recuperado el 2 de octubre de 2026, de https://react.dev